

Import statement

→ 1 from math import pi

→ 2 tau = 2 * pi

Assignment statement

Global frame

Name pi

Value 3.1416

Binding

Code (left):

Statements and expressions
Red arrow points to next line.
Gray arrow points to the line just executed

Frames (right):

A name is bound to a value

In a frame, there is at most one binding per name

1 from operator import mul

2 def square(x):

→ 3 return mul(x, x)

4 square(-2)

Built-in function

func mul(...) [parent=Global]

func square(x) [parent=Global]

User-defined function

Global frame

Intrinsic name of function called

mul

square

Local frame

f1: square [parent=Global]

Formal parameter bound to argument

x -2

Return value 4

Return value is not a binding!

1 from operator import mul

→ 2 def square(x):

→ 3 return mul(x, x)

4 square(square(3))

Global frame

mul

square

f1: square [parent=Global]

x 3

Return value 9

f2: square [parent=Global]

x 9

Return value 81

A name evaluates to the value bound to that name in the earliest frame of the current environment in which that name is found.

1.Evaluate the operator and operand subexpressions.

2.Apply the function that is the value of the operator subexpression to the arguments that are the values of the operand subexpressions.

Applying user-defined functions:

1.Create a new local frame with the same parent as the function that was applied.

2.Bind the arguments to the function's formal parameter names in that frame.

3.Execute the body of the function in the environment beginning at that frame.

Execution rule for def statements:

1.Create a new function value with the specified name, formal parameters, and function body.

2.Its parent is the first frame of the current environment.

3.Bind the name of the function to the function value in the first frame of the current environment.

Execution rule for assignment statements:

1.Evaluate the expression(s) on the right of the equal sign.

2.Simultaneously bind the names on the left to those values, in the first frame of the current environment.

Execution rule for conditional statements:

Each clause is considered in order.

1.Evaluate the header's expression.

2.If it is a true value, execute the suite, then skip the remaining clauses in the statement.

Evaluation rule for or expressions:

1.Evaluate the subexpression <left>.

2.If the result is a true value v, then the expression evaluates to v.

3.Otherwise, the expression evaluates to the value of the subexpression <right>.

Evaluation rule for and expressions:

1.Evaluate the subexpression <left>.

2.If the result is a false value v, then the expression evaluates to v.

3.Otherwise, the expression evaluates to the value of the subexpression <right>.

Evaluation rule for not expressions:

1.Evaluate <exp>; The value is True if the result is a false value, and False otherwise.

Execution rule for while statements:

1.Evaluate the header's expression.

2.If it is a true value, execute the (whole) suite, then return to step 1.

208

mul(add(2, mul(4, 6)), add(3, 5))

mul

26

add(2, mul(4, 6))

add

2

24

mul(4, 6)

mul

4

6

8

add(3, 5)

add

3

5

Dictionary Methods

>>> food = {"ham":10, "cheese":12}

>>> food["cheese"]

12

>>> "peanuts" in food

False

>>> food["peanuts"] = 7 # adds key-value pair to food dict

>>> "peanuts" in food

True

>>> food["ham"] = food["ham"] + 1

>>> food["ham"]

11

>>> [(key, food[key]) for key in food]

[('ham', 11), ('cheese', 12), ('peanuts', 7)]

List comprehensions

[<map exp> for <name> in <iter exp> if <filter exp>]

Short version: [<map exp> for <name> in <iter exp>]

A combined expression that evaluates to a list using this evaluation procedure:

1. Add a new frame with the current frame as its parent

2. Create an empty result list that is the value of the expression

3. For each element in the iterable value of <iter exp>:

A. Bind <name> to that element in the new frame from step 1

B. If <filter exp> evaluates to a true value, then add the value of <map exp> to the result list

List Methods

>>> lst = [8, 61]

>>> lst.append(10)

>>> lst

[8, 61, 10]

>>> lst.extend([2, 3])

>>> lst

[8, 61, 10, 2, 3]

>>> lst.insert(0, 88)

>>> lst

[88, 8, 61, 10, 2, 3]

>>> lst[1:3]

[8, 61]

>>> lst.pop(0)

88

>>> lst

[8, 61, 10, 2, 3]

>>> lst.remove(61)

>>> lst

[8, 10, 2, 3]

>>> lst.pop()

3

>>> lst

[8, 10, 2]

List Environment Diagram

>>> digits = [1, 8, 2, 8]

>>> len(digits)

4

>>> digits[3]

8

>>> digits[1:]

[8, 2, 8]

>>> [2, 7] + digits * 2

[2, 7, 1, 8, 2, 8, 1, 8, 2, 8]

>>> pairs = [[10, 20], [30, 40]]

>>> pairs[1]

[30, 40]

>>> pairs[1][0]

30

Lists "Aggregate" Methods

>>> lst = [-2, 4, 6]

>>> len(lst)

3

>>> sum(lst)

8

>>> min(lst)

-2

>>> max(lst, key=lambda x: -x)

-2

>>> lst = [(1, 9), (2, 5), (3, 4)]

>>> max(lst, key=lambda y: y[0] * y[1])

(3, 4)

Executing a for statement:

for <name> in <expression>:

<suite>

1. Evaluate the header <expression>, which must yield an iterable value (a list, tuple, iterator, etc.)

2. For each element in that sequence, in order:

A. Bind <name> to that element in the current frame

B. Execute the <suite>

..., -3, -2, -1, 0, 1, 2, 3, 4, ...

range(-2, 2)

Length: ending value - starting value

Element selection: starting value + index

>>> list(range(-2, 2))

[-2, -1, 0, 1]

>>> list(range(4))

[0, 1, 2, 3]

List constructor

Range with a 0 starting value

Miscellaneous Operations

>>> 5 // 3

1

>>> 5 % 3

2

>>> 2 * 3

6

>>> 2 + 3

5

>>> 6 / 3

2.0

>>> min(2, 1, 4, 3)

1

>>> max(2, 1, 4, 3)

4

>>> abs(-2)

2

>>> pow(2, 3)

8

>>> len('word')

4

>>> print(1, 2)

1 2

Functional List Operations

map(func, iterable)

Returns an iterator that applies func to every item of iterable.

filter(func, iterable)

Returns an iterator from elements of iterable for which func returns True.

reduce(func, iterable[, initial])

Apply func of two arguments cumulatively to the items of iterable, from left to right, so as to reduce the iterable to a single value.

If the optional initial value is present, it is placed before the items of the iterable in the calculation.

>>> nums = [1, 2, 3]

>>> list(map(lambda x: x ** 2, nums))

[1, 4, 9]

>>> list(filter(lambda x: x % 2 == 0, nums))

[2]

>>> from functools import reduce

>>> add = lambda x, y: x + y

>>> reduce(add, nums) # 1 + 2 + 3 = 6

6

>>> reduce(add, nums, 10) # 10 + 1 + 2 + 3 = 16

16

Recursive description:

- A **tree** has a root **value** and a list of **branches**
- Each branch is a **tree**
- A tree with zero branches is called a **leaf**

Relative description:

- Each location is a **node**
- Each **node** has a **value**
- One node can be the **parent/child** of another

```
class Tree:
    def __init__(self, label, branches=[]):
        self.label = label
        for branch in branches:
            assert isinstance(branch, Tree)
        self.branches = list(branches)

    def is_leaf(self):
        return not self.branches

def leaves(tree):
    """The leaf labels in a tree."""
    if tree.is_leaf():
        return [tree.label]
    else:
        lst = []
        for b in tree.branches:
            lst.extend(leaves(b))
        return lst

def fib_tree(n):
    if n == 0 or n == 1:
        return Tree(n)
    else:
        left = fib_tree(n - 2)
        right = fib_tree(n - 1)
        fib_n = left.label + right.label
        return Tree(fib_n, [left, right])
```

Built-in `isinstance` function: returns True if `branch` has a class that is or inherits from `Tree`

Exceptions are raised with a `raise` statement.

```
raise <expr>
```

<expr> must evaluate to a subclass of `BaseException` or an instance of one.

```
try:
    <try suite>
except <exception class> as <name>:
    <except suite>
```

The <try suite> is executed first. If, during the course of executing the <try suite>, an exception is raised that is not handled otherwise, and

If the class of the exception inherits from <exception class>, then The <except suite> is executed, with <name> bound to the exception.

```
>>> try:
    x = 1 / 0
except ZeroDivisionError as e:
    print('handling a', type(e))
    x = 0

handling a <class 'ZeroDivisionError'>
>>> x
0
```

```
class Link:
    empty = ()

    def __init__(self, first, rest=empty):
        assert rest is Link.empty or isinstance(rest, Link)
        self.first = first
        self.rest = rest

    def __repr__(self):
        if self.rest:
            rest = ', ' + repr(self.rest)
        else:
            rest = ''
        return 'Link(' + repr(self.first) + rest + ')'

    def __str__(self):
        string = '('
        while self.rest is not Link.empty:
            string += str(self.first) + ', '
            self = self.rest
        return string + str(self.first) + ')'
```

Some zero length sequence

```
>>> s = Link(4, Link(5))
>>> s
Link(4, Link(5))
>>> s.first
4
>>> s.rest
Link(5)
>>> print(s)
(4 5)
>>> print(s.rest)
(5)
>>> s.rest.rest is Link.empty
True
```

Anatomy of a recursive function:

- The **def statement header** is like any function
- Conditional statements check for **base cases**
- Base cases are evaluated **without recursive calls**
- Recursive cases are evaluated **with recursive calls**

```
def sum_digits(n):
    """Sum the digits of positive integer n."""
    if n < 10:
        return n
    else:
        all_but_last, last = n // 10, n % 10
        return sum_digits(all_but_last) + last
```

- Recursive decomposition:** finding simpler instances of a problem.
- E.g., `count_partitions(6, 4)`
- Explore two possibilities:
 - Use at least one 4
 - Don't use any 4
- Solve two simpler problems:
 - `count_partitions(2, 4)`
 - `count_partitions(6, 3)`
- Tree recursion often involves exploring different choices.

```
def count_partitions(n, m):
    if n == 0:
        return 1
    elif n < 0:
        return 0
    elif m == 0:
        return 0
    else:
        with_m = count_partitions(n-m, m)
        without_m = count_partitions(n, m-1)
        return with_m + without_m
```

Python object system:

Idea: All bank accounts have a **balance** and an account **holder**; the **Account** class should add those attributes to each of its instances

```
>>> a = Account('Jim')
>>> a.holder
'Jim'
>>> a.balance
0
```

A new instance is created by calling a class

When a class is called:

- 1.A new instance of that class is created:
- 2.The `__init__` method of the class is called with the new object as its first argument (named `self`), along with any additional arguments provided in the call expression.

```
class Account:
    def __init__(self, account_holder):
        self.balance = 0
        self.holder = account_holder
    def deposit(self, amount):
        self.balance = self.balance + amount
        return self.balance
    def withdraw(self, amount):
        if amount > self.balance:
            return 'Insufficient funds'
        self.balance = self.balance - amount
        return self.balance
```

`__init__` is called a constructor

self should always be bound to an instance of the Account class or a subclass of Account

```
>>> type(Account.deposit)
<class 'function'>
>>> type(a.deposit)
<class 'method'>
```

Function call: all arguments within parentheses

Method invocation: One object before the dot and other arguments within parentheses

```
>>> Account.deposit(a, 5)
10
>>> a.deposit(2)
12
```

Call expression

Dot expression

`<expression> . <name>`

The `<expression>` can be any valid Python expression. The `<name>` must be a simple name. Evaluates to the value of the attribute looked up by `<name>` in the object that is the value of the `<expression>`.

To evaluate a dot expression:

1. Evaluate the `<expression>` to the left of the dot, which yields the object of the dot expression
2. `<name>` is matched against the instance attributes of that object; if an attribute with that name exists, its value is returned
3. If not, `<name>` is looked up in the class, which yields a class attribute value
4. That value is returned unless it is a function, in which case a bound method is returned instead

Assignment statements with a dot expression on their left-hand side affect attributes for the object of that dot expression

- If the object is an instance, then assignment sets an instance attribute
- If the object is a class, then assignment sets a class attribute

```
>>> jim_account = Account('Jim')
>>> tom_account = Account('Tom')
>>> tom_account.interest
0.02
>>> jim_account.interest
0.02
>>> Account.interest = 0.04
>>> tom_account.interest
0.04
>>> jim_account.interest
0.04
```

Account class attributes

Instance attributes of jim_account

Instance attributes of tom_account

```
>>> jim_account.interest = 0.08
>>> jim_account.interest
0.08
>>> tom_account.interest
0.04
>>> Account.interest = 0.05
>>> tom_account.interest
0.05
>>> jim_account.interest
0.08
```

```
class CheckingAccount(Account):
    """A bank account that charges for withdrawals."""
    withdraw_fee = 1
    interest = 0.01
    def withdraw(self, amount):
        return Account.withdraw(self, amount + self.withdraw_fee)
        or
        return super().withdraw(amount + self.withdraw_fee)
```

To look up a name in a class:

1. If it names an attribute in the class, return the attribute value.
2. Otherwise, look up the name in the base class, if there is one.

```
>>> ch = CheckingAccount('Tom') # Calls Account.__init__
>>> ch.interest # Found in CheckingAccount
0.01
>>> ch.deposit(20) # Found in Account
20
>>> ch.withdraw(5) # Found in CheckingAccount
14
```

```
iter(iterable):           >>> s = [3, 4, 5] >>> d = {'one': 1, 'two': 2, 'three': 3}
    Return an iterator    >>> t = iter(s) >>> k = iter(d)
    over the elements of >>> next(t) >>> next(k) >>> v = iter(d.values())
    an iterable value     3 >>> next(t) >>> next(k) >>> next(v)
next(iterator):           >>> next(t) >>> next(k) >>> next(v)
    Return the next element 4 3 'one' 1
                           4 4 'two' 2

A generator function is a function that yields values instead of returning.
>>> def plus_minus(x): >>> t = plus_minus(3) def a_then_b(a, b):
...     yield x         >>> next(t)         yield from a
...     yield -x        >>> next(t)         yield from b
                           3 >>> next(t)         >>> list(a_then_b([3, 4], [5, 6]))
                           -3 [3, 4, 5, 6]
```

Efficiency

Constant growth. E.g., accessing a value from a dictionary.
Increasing *n* doesn't affect time

Logarithmic growth. E.g., binary search
Doubling *n* only increments *time* by a constant

Linear growth. E.g., iterating over a list of length *n*
Incrementing *n* increases *time* by a constant

Quadratic growth. E.g., finding all pairs of a list of integers (double for loop)
Incrementing *n* increases *time* by *n* times a constant

Exponential growth. E.g., recursive `fib`
Incrementing *n* multiplies *time* by a constant

$O(1)$

$O(\log n)$

$O(n)$

$O(n^2)$

$O(b^n)$

```
def perms(lst):
    """Generates the permutations of lst one by one.
    >>> perms = perms([1, 2, 3])
    >>> p = list(perms)
    >>> p.sort()
    >>> p
    [[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]
    """
    if lst == []:
        yield []
    else:
        for perm in perms(lst[1:]):
            for i in range(len(lst)):
                yield perm[:i] + [lst[0]] + perm[i:]
```

A table has columns and rows

Latitude	Longitude	Name
38	122	Berkeley
42	71	Cambridge
45	93	Minneapolis

A row has a value for each column

A column has a name and a type

```
SELECT [expression] AS [name], [expression] AS [name], ... ;
```

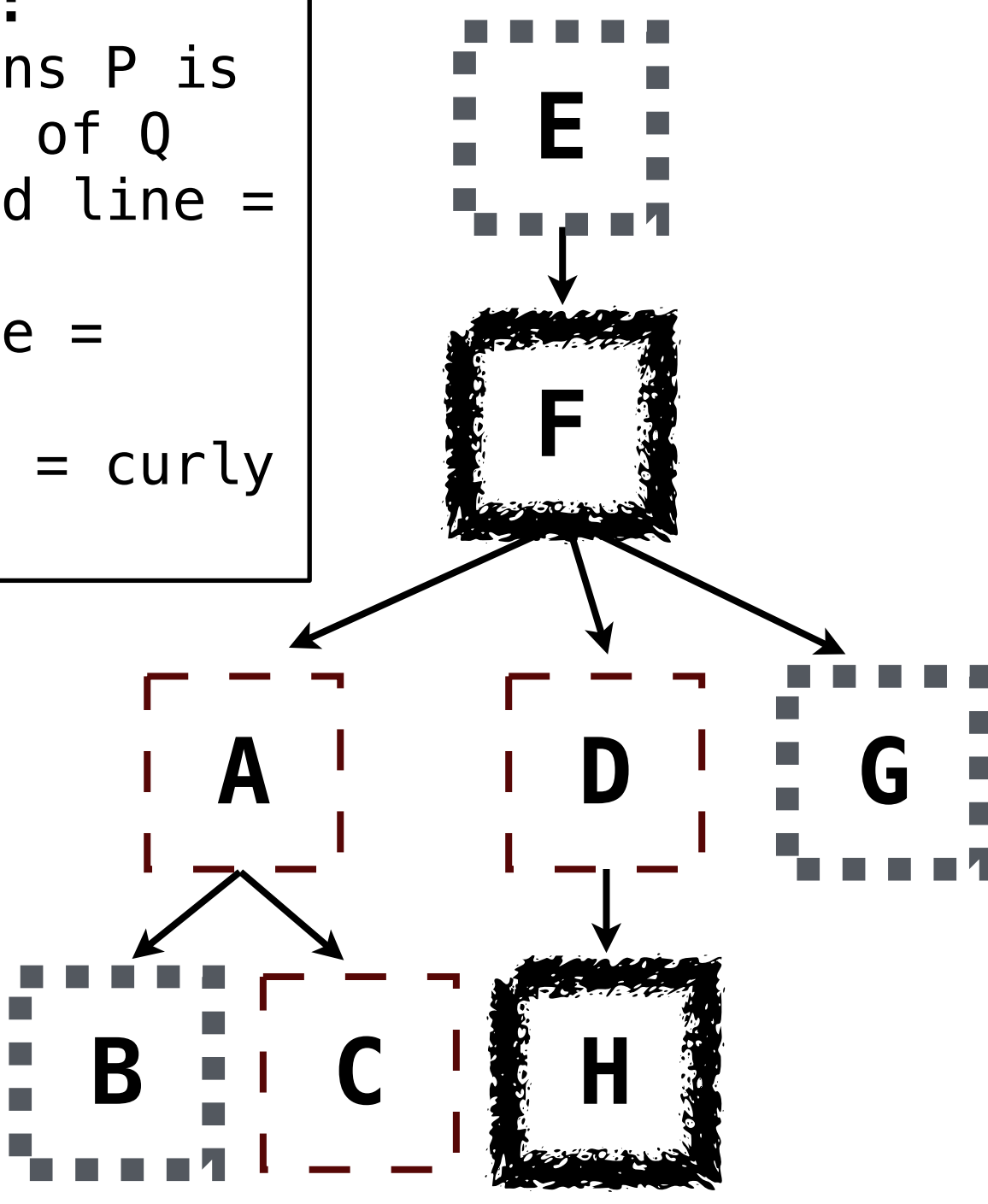
```
SELECT [columns] FROM [table] WHERE [condition] ORDER BY [order];
```

```
CREATE TABLE parents AS
SELECT "D" AS parent, "H" AS child UNION
SELECT "A"           , "B"           UNION
SELECT "A"           , "C"           UNION
SELECT "F"           , "A"           UNION
SELECT "F"           , "D"           UNION
SELECT "F"           , "G"           UNION
SELECT "E"           , "F";

CREATE TABLE dogs AS
SELECT "A" AS name, "long" AS fur UNION
SELECT "B"           , "short"      UNION
SELECT "C"           , "long"       UNION
SELECT "D"           , "long"       UNION
SELECT "E"           , "short"      UNION
SELECT "F"           , "curly"      UNION
SELECT "G"           , "short"      UNION
SELECT "H"           , "curly";
```

Diagram Key:

- P -> Q means P is the parent of Q
- Thin dashed line = long fur
- Dotted line = short fur
- Thick line = curly fur



first	second
B	C
A	D
A	G
D	G

```
SELECT a.child AS first, b.child AS second
FROM parents AS a, parents AS b
WHERE a.parent = b.parent AND a.child < b.child;
```

String values can be combined to form longer strings

```
sqlite> SELECT "hello," || " world";
hello, world
```

Basic string manipulation is built into SQL, but differs from Python

```
sqlite> CREATE TABLE phrase AS SELECT "hello, world" AS s;
sqlite> SELECT substr(s, 4, 2) || substr(s, instr(s, " ") + 1, 1)
FROM phrase;
low
```

The number of groups is the number of unique values of an expression
A **HAVING** clause filters the set of groups that are aggregated

```
SELECT weight / legs, COUNT(*) FROM animals
GROUP BY weight / legs
HAVING COUNT(*) > 1;
```

weight/legs	COUNT(*)
5	2
2	2

- weight/legs=5
- weight/legs=2
- weight/legs=2
- weight/legs=3
- weight/legs=5
- weight/legs=6000

kind	legs	weight
dog	4	20
cat	4	10
ferret	4	10
parrot	2	6
penguin	2	10
t-rex	2	12000

An aggregate function in the [columns] clause computes a value from a group of rows:

- MAX([expression]) evaluates to the largest value of [expression] for any row in a group
- COUNT(*) evaluates to the number of rows in a group
- MIN, SUM, & AVG are also aggregate functions similar to MAX

With no GROUP BY clause, aggregation is performed over all rows:

```
SELECT MAX(legs) FROM animals;
```

MAX(legs)
4